

JurisAI - Setup & Run Guide

Smart Legal Advisor & Document Assistant

A React (frontend) + Django REST (backend) web app with four AI modules: Legal Advisor chatbot, Document Summarization, Multilingual Document Generation, and Agreement Clause Verification. AI is powered by Google Gemini (free tier), with built-in offline fallbacks.

1. What you will run

- Backend API (Django) -> `http://localhost:8000`
- Frontend app (React) -> `http://localhost:5173`
- You open the frontend in a browser; it talks to the backend API.

2. Prerequisites (install these first)

- Node.js 18 or newer (check with: `node --version`)
- Python 3.10 - 3.12 (check with: `python --version`)
- Git (check with: `git --version`)

Downloads: Node -> <https://nodejs.org> | Python -> <https://python.org/downloads>

IMPORTANT: on the Python installer, tick 'Add python.exe to PATH'.

3. Get the code from GitHub

```
git clone <YOUR-REPO-URL> jurisai
cd jurisai
```

Replace <YOUR-REPO-URL> with your repository URL. After cloning you will have two folders: `backend/` and `frontend/`.

4. Backend setup (Django API)

Open a terminal in the project folder and run these one block at a time:

```
cd backend

# create and activate a virtual environment
python -m venv venv
venv\Scripts\activate # Windows (PowerShell / CMD)
# source venv/bin/activate # macOS / Linux

# install all Python dependencies
pip install -r requirements.txt

# create the database tables (SQLite, no setup needed)
python manage.py makemigrations
python manage.py migrate

# (optional) create an admin login for /admin + Admin console
python manage.py createsuperuser

# start the API server
python manage.py runserver
```

Leave this terminal running. The API is now at `http://localhost:8000` .

5. Add your Gemini API key (for live AI)

The backend reads AI settings from `backend/.env` . This file is kept out of git

for security, so create it from the template after cloning:

```
# inside the backend/ folder
copy .env.example .env # Windows
# cp .env.example .env # macOS / Linux
```

Then open backend/.env and set your free Google Gemini key:

```
LLM_PROVIDER=auto
GEMINI_API_KEY=<paste-your-gemini-key-here>
GEMINI_MODEL=gemini-2.0-flash
```

Get a free key at <https://aistudio.google.com/apikey> (a valid key usually starts with 'AIza'). Restart the server after editing .env. WITHOUT a key the app still works using built-in offline answers and summaries.

6. Frontend setup (React)

Open a SECOND terminal (keep the backend running in the first one):

```
cd frontend
npm install
npm run dev
```

This starts the app at <http://localhost:5173> and usually opens it automatically. If the backend is not running, the frontend shows sample 'demo' data so every screen still works.

7. Use the app

- Open <http://localhost:5173> in your browser.
- Create an account (Full name, Email, Password), then log in.
- Home: choose a module - Legal Advisor, Document Summary, Generate Document, or Clause Verification.
- Legal Advisor: type a legal question and get an answer with citations.
- Document Summary: upload a PDF/DOCX/TXT to get a structured summary + Q&A.
- Generate Document: pick a template or describe a custom draft, choose a language, download as PDF.
- Clause Verification: upload a contract to see a risk score, flagged loopholes and corrected wording.

8. Troubleshooting

- 'python' is not recognized: reinstall Python with 'Add to PATH' ticked, or use 'py' instead of 'python'.
- pip fails building PyMuPDF: upgrade pip first -> `python -m pip install --upgrade pip`
- Port already in use: run the API on another port -> `python manage.py runserver 8001` (then set frontend/.env: `VITE_API_BASE_URL=http://localhost:8001/api`).
- Browser network/CORS errors: ensure the backend is running and <http://localhost:5173> is listed in backend/.env `CORS_ALLOWED_ORIGINS`.
- Answers look generic / provider 'offline': the Gemini key is missing or invalid
 - make a new one at aistudio.google.com/apikey and update backend/.env.
- 'Demo mode' banner appears: the backend is not reachable - start it with `python manage.py runserver`.

9. Project structure

```
jurisai/
backend/ Django REST API
legal_ai/ settings & root URLs
common/llm.py AI layer (Gemini / OpenAI / Anthropic)
```

authentication/ JWT login & registration
chatbot/ Legal Advisor
summarizer/ document summarization + Q&A
document_generation/ templates, drafting, PDF export
clause_verification/ clause loophole / risk analysis
file_manager/ uploads + text extraction
requirements.txt .env.example
frontend/ React (Vite) app
src/pages/ src/components/ src/api/

10. Important notes

- Never commit your real API key. backend/.env is already git-ignored.
- Default AI provider is Google Gemini (free); OpenAI and Anthropic also work via .env.
- JurisAI provides general legal information, not legal advice.